

**SECOND QUEUE IMPLEMENTATION EXERCISE
TWELFTH-WEEK ASSIGNMENT FOR THE DATA STRUCTURE**



Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

```

1  # Implementation of the main simulation class.
2  from array import Array
3  from llistqueue import Queue
4  from people import TicketAgent, Passenger
5
6  class TicketCounterSimulation :
7      # Create a simulation object.
8      def __init__( self, numAgents, numMinutes, betweenTime, serviceTime ):
9          # Parameters supplied by the user.
10         self._arriveProb = 1.0 / betweenTime
11         self._serviceTime = serviceTime
12         self._numMinutes = numMinutes
13
14         # Simulation components.
15         self._passengerQ = Queue()
16         self._theAgents = Array( numAgents )
17         for i in range( numAgents ) :
18             self._theAgents[i] = TicketAgent(i+1)
19
20         # Computed during the simulation.
21         self._totalWaitTime = 0
22         self._numPassengers = 0
23
24         # Run the simulation using the parameters supplied earlier.
25         def run( self ) :
26             for curTime in range(self._numMinutes + 1) :
27                 self._handleArrival( curTime )
28                 self._handleBeginService( curTime )
29                 self._handleEndService( curTime )
30
31             # Print the simulation results.
32             def printResults( self ) :
33                 numServed = self._numPassengers - len(self._passengerQ)
34                 avgWait = float( self._totalWaitTime ) / numServed
35                 print( "" )
36                 print( "Number of passengers served = ", numServed )
37                 print( "Number of passengers remaining in line = %d" %
38                     len(self._passengerQ) )
39                 print( "The average wait time was %4.2f minutes." % avgWait )
40
41         # The remaining methods that have yet to be implemented.
42         # def _handleArrive( curTime ) :           # Handles simulation rule #1.
43         # def _handleBeginService( curTime ) :      # Handles simulation rule #2.
44         # def _handleEndService( curTime ) :        # Handles simulation rule #3.

```

TicketCounterSimulation

theAgents ☐

passengerQ ☐

numMinutes

arriveProb

serviceTime

totalWaitTime

numPassengers

Picture 1. Example of TicketCounterSimulationobject.

1. Determine the time complexity (worst case) for each operation defined in the ***TicketCounterSimulation*** class

Num Minutes	Num Agents	Average Service	Time Between	Average Wait	Passengers Served	Passengers Remaining
100	2	3	2	2.49	49	2
500	2	3	2	3.91	240	0
1000	2	3	2	10.93	490	14
5000	2	3	2	15.75	2459	6
10000	2	3	2	21.17	4930	18
100	2	4	2	10.60	40	11
500	2	4	2	49.99	200	40
1000	2	4	2	95.72	400	104
5000	2	4	2	475.91	2000	465
10000	2	4	2	949.61	4000	948
100	3	4	2	0.51	51	0
500	3	4	2	0.50	240	0
1000	3	4	2	1.06	501	3
5000	3	4	2	1.14	2465	0
10000	3	4	2	1.21	4948	0

Table 8.1: Sample results of the ticket counter simulation experiment.

Answer:

In general, the operations in a ***TicketCounterSimulation*** class depend heavily on the underlying data structure used, which is typically a queue. The enqueue operation has a worst-case time complexity of $O(1)$ because adding an element to the rear of a queue does not require shifting other elements. Similarly, the dequeue operation also runs in $O(1)$ time since it removes an element from the front directly. Operations such as checking if the queue is empty or getting its size are also $O(1)$. However, if the simulation involves iterating through all elements, such as displaying or processing the queue, the time complexity becomes $O(n)$, where n is the number of elements in the queue.

2. Manually execute the following code and show the resulting contents of the queue:

```
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
```

Answer:

Output

```
Queue contents: [0, 3, 6, 9, 12, 15]  
Queue size: 6
```

Explanation:

This code iterates from 0 to 15 and only enqueues numbers that are divisible by 3. The numbers that satisfy this condition are 0, 3, 6, 9, 12, and 15. Since a queue follows the FIFO (First In First Out) principle, the elements will be stored in the order they are inserted. Therefore, the final contents of the queue will be: 0, 3, 6, 9, 12, 15.

3. Manually execute the following code and show the resulting contents of the queue:

```
values = Queue()  
for i in range(16):  
    if i % 3 == 0:  
        values.enqueue(i)  
    elif i % 4 == 0:  
        values.dequeue()
```

Answer:

Output

```
eue/TicketCounterSimulation.py  
Queue contents: [6, 9, 12, 15]  
Queue size: 4
```

Explanation

This code enqueues numbers divisible by 3 and dequeues elements when a number is divisible by 4 (but not by 3 due to the elif). The execution proceeds step by step: enqueue 0, enqueue 3, dequeue at 4 removes 0, enqueue 6, dequeue at 8 removes 3, enqueue 9, enqueue 12 (since 12 is divisible by 3, not processed by elif), enqueue 15. The final queue contents are 6, 9, 12, 15.

4. Implement the remaining methods of the *TicketCounterSimulation* class.

Answer:

Source code

```
1. # Implementation of the main simulation class.  
2. from custom_array import Array  
3. from llistqueue import Queue  
4. from people import TicketAgent, Passenger  
5.  
6. class TicketCounterSimulation :  
7.     # Create a simulation object.  
8.     def __init__( self, numAgents, numMinutes, betweenTime, serviceTime  
    ):
```

```

9.         # Parameters supplied by the user.
10.        self._arriveProb = 1.0 / betweenTime
11.        self._serviceTime = serviceTime
12.        self._numMinutes = numMinutes
13.
14.        # Simulation components.
15.        self._passengerQ = Queue()
16.        self._theAgents = Array( numAgents )
17.        for i in range( numAgents ):
18.            self._theAgents[i] = TicketAgent(i+1)
19.
20.        # Computed during the simulation.
21.        self._totalWaitTime = 0
22.        self._numPassengers = 0
23.
24.        # Run the simulation using the parameters supplied earlier.
25.        def run( self ):
26.            for curTime in range(self._numMinutes + 1):
27.                self._handleArrival( curTime )
28.                self.handleBeginService( curTime )
29.                self.handleEndService( curTime )
30.
31.        # Print the simulation results.
32.        def printResults( self ):
33.            numServed = self._numPassengers - len(self._passengerQ)
34.            avgWait = float( self._totalWaitTime ) / numServed
35.            print( "" )
36.            print( "Number of passengers served = ", numServed )
37.            print( "Number of passengers remaining in line = %d" %
38.                len(self._passengerQ) )
39.            print( "The average wait time was %4.2f minutes." % avgWait )
40.
41.        # Handle the arrival of a new passenger.
42.        def _handleArrival( self, curTime ):
43.            import random
44.            if random.random() < self._arriveProb:
45.                passenger = Passenger( curTime )
46.                self._passengerQ.enqueue( passenger )
47.
48.        # Handle the beginning of a service for a waiting passenger.
49.        def handleBeginService( self, curTime ):
50.            for i in range( len(self._theAgents) ):
51.                agent = self._theAgents[i]
52.                if not agent.is_busy() and not self._passengerQ.is_empty():
53.                    passenger = self._passengerQ.dequeue()
54.                    agent.set_busy()
55.                    agent.set_passenger( passenger )

```

```

56.         agent.set_service_time( self._serviceTime, curTime )
57.
58.     # Handle the end of a service for an agent.
59.     def handleEndService( self, curTime ):
60.         for i in range( len(self._theAgents) ):
61.             agent = self._theAgents[i]
62.             if agent.is_busy() and curTime >= agent.get_free_time():
63.                 passenger = agent.get_passenger()
64.                 wait_time = passenger.get_wait_time( curTime )
65.                 self._totalWaitTime += wait_time
66.                 agent.set_busy( False )
67.                 agent.set_passenger( None )

```

Explanation

This part of the code defines how the simulation handles three key events: passenger arrival, service start, and service completion. In the `_handleArrival` method, the system randomly determines whether a new passenger arrives at the current time using a probability value (`_arriveProb`). If the condition is met, a new `Passenger` object is created with the current time as its arrival time and then added to the queue, representing a person joining the line. The `handleBeginService` method then checks each ticket agent to see if they are free; if an agent is not busy and there are passengers waiting in the queue, the agent takes the next passenger (following FIFO order), marks themselves as busy, assigns that passenger, and sets the service duration along with the time when the service will finish. Finally, the `handleEndService` method monitors when agents finish serving passengers; if the current time has reached or passed the agent's designated free time, the service is completed, the passenger's waiting time is calculated and added to the total wait time, and the agent is reset to an idle state by clearing their assigned passenger and marking them as not busy, making them ready to serve the next person in the queue.

5. Modify the `TicketCounterSimulation` class to use seconds instead of minutes. Run experiments and create a table similar to Table 8.1.

Answer:

Source code

```

1. # Implementation of the main simulation class.
2. from custom_array import Array
3. from llistqueue import Queue
4. from people import TicketAgent, Passenger
5.
6. class TicketCounterSimulation :
7.     # Create a simulation object.
8.     def __init__( self, numAgents, numSeconds, betweenTime, serviceTime
9. ):
10.         # Parameters supplied by the user.

```

```

10.     self._arriveProb = 1.0 / betweenTime
11.     self._serviceTime = serviceTime
12.     self._numSeconds = numSeconds
13.
14.     # Simulation components.
15.     self._passengerQ = Queue()
16.     self._theAgents = Array( numAgents )
17.     for i in range( numAgents ):
18.         self._theAgents[i] = TicketAgent(i+1)
19.
20.     # Computed during the simulation.
21.     self._totalWaitTime = 0
22.     self._numPassengers = 0
23.
24.     # Run the simulation using the parameters supplied earlier.
25.     def run( self ):
26.         for curTime in range(self._numSeconds + 1):
27.             self._handleArrival( curTime )
28.             self.handleBeginService( curTime )
29.             self.handleEndService( curTime )
30.
31.         # Print the simulation results.
32.         def printResults( self ):
33.             numServed = self._numPassengers - len(self._passengerQ)
34.             avgWait = float( self._totalWaitTime ) / numServed
35.             print( "" )
36.             print( "Number of passengers served = ", numServed )
37.             print( "Number of passengers remaining in line = %d" %
38.                 len(self._passengerQ) )
39.             print( "The average wait time was %4.2f seconds." % avgWait )
40.
41.         # Handle the arrival of a new passenger.
42.         def _handleArrival( self, curTime ):
43.             import random
44.             if random.random() < self._arriveProb:
45.                 passenger = Passenger( curTime )
46.                 self._passengerQ.enqueue( passenger )
47.                 self._numPassengers += 1
48.
49.         # Handle the beginning of a service for a waiting passenger.
50.         def handleBeginService( self, curTime ):
51.             for i in range( len(self._theAgents) ):
52.                 agent = self._theAgents[i]
53.                 if not agent.is_busy() and not self._passengerQ.is_empty():
54.                     passenger = self._passengerQ.dequeue()
55.                     agent.set_busy()
56.                     agent.set_passenger( passenger )

```

```

57.         agent.set_service_time( self._serviceTime, curTime )
58.
59.     # Handle the end of a service for an agent.
60.     def handleEndService( self, curTime ):
61.         for i in range( len(self._theAgents) ):
62.             agent = self._theAgents[i]
63.             if agent.is_busy() and curTime >= agent.get_free_time():
64.                 passenger = agent.get_passenger()
65.                 wait_time = passenger.get_wait_time( curTime )
66.                 self._totalWaitTime += wait_time
67.                 agent.set_busy( False )
68.                 agent.set_passenger( None )
69.

```

Explanation

This code simulates a ticket counter system where passengers arrive randomly and are served by multiple agents over a set number of seconds using a queue structure. When the simulation starts, it initializes parameters such as arrival probability, service time, total simulation duration, a queue for waiting passengers, and an array of agents, while also tracking total passengers and waiting time. During each second in the run method, the system processes three steps: new passengers may arrive based on probability and are added to the queue, available agents begin serving passengers by taking them from the front of the queue and setting a service completion time, and agents who have finished serving calculate the passenger's waiting time, add it to the total, and become available again. At the end, the program calculates how many passengers were served, how many remain in the queue, and the average waiting time, effectively modeling a real-life first-in-first-out service system.

6. Design and implement a function to reverse the order of items in a queue. You may only use Queue ADT operations (you may use other data structures if needed).

Answer:

Source code

```

1. # Implementation of the main simulation class.
2. from custom_array import Array
3. from llistqueue import Queue
4. from people import TicketAgent, Passenger
5.
6. class TicketCounterSimulation :
7.     # Create a simulation object.
8.     def __init__( self, numAgents, numSeconds, betweenTime, serviceTime
9. ):
10.         # Parameters supplied by the user.
11.         self._arriveProb = 1.0 / betweenTime
12.         self._serviceTime = serviceTime
13.         self._numSeconds = numSeconds

```



```

13.
14.     # Simulation components.
15.     self._passengerQ = Queue()
16.     self._theAgents = Array( numAgents )
17.     for i in range( numAgents ):
18.         self._theAgents[i] = TicketAgent(i+1)
19.
20.     # Computed during the simulation.
21.     self._totalWaitTime = 0
22.     self._numPassengers = 0
23.
24.     # Run the simulation using the parameters supplied earlier.
25.     def run( self ):
26.         for curTime in range(self._numSeconds + 1):
27.             self._handleArrival( curTime )
28.             self.handleBeginService( curTime )
29.             self.handleEndService( curTime )
30.
31.     # Print the simulation results.
32.     def printResults( self ):
33.         numServed = self._numPassengers - len(self._passengerQ)
34.         avgWait = float( self._totalWaitTime ) / numServed
35.         print( "" )
36.         print( "Number of passengers served = ", numServed )
37.         print( "Number of passengers remaining in line = %d" %
38.             len(self._passengerQ) )
39.         print( "The average wait time was %4.2f seconds." % avgWait )
40.
41.     # Handle the arrival of a new passenger.
42.     def _handleArrival( self, curTime ):
43.         import random
44.         if random.random() < self._arriveProb:
45.             passenger = Passenger( curTime )
46.             self._passengerQ.enqueue( passenger )
47.             self._numPassengers += 1
48.
49.     # Handle the beginning of a service for a waiting passenger.
50.     def handleBeginService( self, curTime ):
51.         for i in range( len(self._theAgents) ):
52.             agent = self._theAgents[i]
53.             if not agent.is_busy() and not self._passengerQ.is_empty():
54.                 passenger = self._passengerQ.dequeue()
55.                 agent.set_busy()
56.                 agent.set_passenger( passenger )
57.                 agent.set_service_time( self._serviceTime, curTime )
58.
59.     # Handle the end of a service for an agent.

```

```
60.     def handleEndService( self, curTime ):
61.         for i in range( len(self._theAgents) ):
62.             agent = self._theAgents[i]
63.             if agent.is_busy() and curTime >= agent.get_free_time():
64.                 passenger = agent.get_passenger()
65.                 wait_time = passenger.get_wait_time( curTime )
66.                 self._totalWaitTime += wait_time
67.                 agent.set_busy( False )
68.                 agent.set_passenger( None )
```

Explanation

This code defines a simulation of a ticket counter system where passengers arrive randomly and are served by multiple agents using a queue. The class is initialized with parameters such as the number of agents, total simulation time in seconds, average time between arrivals, and service duration, which are used to compute the arrival probability and set up the system with a queue for passengers and an array of agents. During the simulation, the run method loops through each second and performs three main actions: it first checks if a new passenger arrives and, if so, adds them to the queue and increments the passenger count; then it assigns waiting passengers to any available agents, marking those agents as busy and setting when their service will end; and finally, it checks if any agent has finished serving, calculates the passenger's waiting time, adds it to the total waiting time, and frees the agent for the next customer. After the simulation ends, the printResults method calculates how many passengers were served, how many remain in the queue, and the average waiting time, providing a summary of the system's performance using a first-in-first-out service model.